

DSPY Boss System - Complete Cloning Prompt

Create a sophisticated autonomous AI system dashboard called “DSPY Boss” that combines a Next.js frontend with a Python backend, using DSPY signatures for intelligent decision-making and agent coordination.

Project Overview

Build a fully autonomous AI system that operates without manual intervention, featuring:

- **Autonomous Execution Engine** with continuous operation
- **Hierarchical Agent System** (Boss Agent + Subordinates)
- **State Management** with historical storage and forecasting
- **Multi-LLM Provider Support** with failover
- **Real-time Dashboard** with luxury glass UI design
- **MCP Server Integration** for external services
- **Chat Interface** to communicate with the Boss Agent

Project Structure

Create the following directory structure:

```

dspy_boss_dashboard/
├── README.md
├── requirements.txt
├── .gitignore
├── app/
│   ├── package.json
│   ├── next.config.js
│   ├── tailwind.config.ts
│   ├── tsconfig.json
│   ├── components.json
│   ├── prisma/
│   │   └── schema.prisma
│   └── app/
│       ├── layout.tsx
│       ├── page.tsx
│       ├── globals.css
│       ├── agents/page.tsx
│       ├── boss/page.tsx
│       ├── config/page.tsx
│       ├── health/page.tsx
│       ├── logs/page.tsx
│       ├── tasks/page.tsx
│       ├── performance/page.tsx
│       ├── mcp-servers/page.tsx
│       └── failures/page.tsx
│   └── api/
│       ├── auth/
│       ├── agents/route.ts
│       ├── tasks/route.ts
│       ├── system/
│       └── mcp-servers/
├── components/
│   ├── ui/
│   ├── sidebar.tsx
│   ├── header.tsx
│   ├── dashboard-card.tsx
│   ├── system-overview-dashboard.tsx
│   ├── agent-management.tsx
│   ├── boss-state-manager.tsx
│   ├── chat-with-boss.tsx
│   ├── task-management.tsx
│   ├── mcp-server-status.tsx
│   └── theme-provider.tsx
├── lib/
│   ├── utils.ts
│   ├── types.ts
│   ├── api-client.ts
│   ├── dashboard-utils.ts
│   ├── db.ts
│   └── mock-data.ts
├── hooks/
│   └── use-toast.ts
└── backend/
    ├── integration.py
    ├── autonomous/
    │   ├── __init__.py
    │   ├── autonomous_executor.py
    │   ├── execution_cycle.py
    │   └── iteration_engine.py
    ├── agents/
    │   ├── __init__.py
    │   └── agent_hierarchy.py

```

Next.js Frontend

App Router Pages

Root layout

Dashboard home

Global styles

Agent management

Boss chat **interface**

System configuration

System health

System logs

Task management

Performance metrics

MCP server status

Failure analysis

API routes

Authentication

Agent management

Task operations

System endpoints

MCP endpoints

React Components

Shadcn UI components

Navigation sidebar

Page header

Dashboard cards

Utilities

Python Backend

Main integration layer

Autonomous execution

Continuous operation

Cycle management

Iteration logic

Agent management

Boss + subordinates

- **UI Components:** Radix UI + Shadcn components
- **State Management:** Zustand for client state
- **Data Fetching:** SWR for server state
- **Authentication:** NextAuth.js with Prisma
- **Charts:** Recharts and Chart.js for analytics
- **Icons:** Lucide React icons

2. Backend Stack (Python)

- **Core Framework:** DSPY for AI signatures
- **Async Framework:** asyncio for concurrent operations
- **Database:** SQLite for state storage
- **Logging:** Loguru for structured logging
- **Validation:** Pydantic for data models
- **HTTP Client:** aiohttp for external APIs
- **Container Support:** Docker for Ollama integration

3. Key Features to Implement

A. Autonomous Execution Engine

```
class AutonomousExecutor:
    # Continuous operation without manual triggers
    # Configurable iteration intervals (default: 30 seconds)
    # Error handling with exponential backoff
    # Adaptive timing based on performance
```

B. Agent Hierarchy System

```
class AgentHierarchy:
    # Boss Agent (Agent 0) - Supreme decision maker
    # Subordinate Agents (Agent 1, 2, 3...) - Specialized workers
    # Dynamic agent spawning based on workload
    # Inter-agent communication system
```

C. State Management

```
class StateHolder:
    # Real-time state tracking (last 100 states)
    # Historical state storage with compression
    # Pattern recognition and analysis
    # Performance forecasting with ML
```

D. LLM Provider Management

```
class LLMProviderManager:
    # Multi-provider support (OpenAI, Grok, Google, OpenRouter, Ollama)
    # Load balancing strategies
    # Automatic failover
    # Usage tracking and optimization
```

Detailed Implementation Steps

Phase 1: Project Setup

1. Create Next.js project with App Router
2. Install all frontend dependencies from package.json
3. Set up Tailwind CSS with luxury dark theme
4. Create Python backend directory structure
5. Install Python dependencies from requirements.txt

Phase 2: Backend Core Systems

1. **DSPY Signatures:** Create signature classes for:
 - Agent spawning decisions
 - Task delegation
 - State forecasting
 - System reflection
 - Decision making
2. **State Management:** Implement:
 - StateHolder for current state
 - HistoricalStateManager for long-term storage
 - StateForecaster for predictions
 - PersistenceLayer for data compression
3. **Agent System:** Build:
 - AgentHierarchy with Boss + subordinates
 - AgentSpawner for intelligent agent creation
 - AgentManager for coordination
 - AgentCommunicationHub for messaging
4. **LLM Providers:** Create:
 - Individual provider classes (OpenAI, Grok, etc.)
 - ProviderManager for coordination
 - ConfigManager for API keys
 - Load balancing and failover logic

Phase 3: Autonomous Execution

1. **Iteration Engine:** Core iteration lifecycle
2. **Autonomous Executor:** Continuous operation loop
3. **Execution Cycle:** System coordination
4. **Integration Layer:** Frontend-backend bridge

Phase 4: Frontend Dashboard

1. **Layout System:** Root layout with sidebar navigation
2. **Dashboard Components:**
 - System overview with real-time metrics
 - Agent status grid with health indicators
 - Task management interface
 - Performance charts and analytics

- Boss chat interface
- MCP server status

3. **UI Components:** Luxury glass design with:

- Glassmorphism effects
- Dark theme with healthy accents
- Smooth animations
- Responsive design

Phase 5: Advanced Features

1. **Chat Interface:** Direct communication with Boss Agent
2. **MCP Integration:** News and YouTube analytics
3. **Authentication:** User management system
4. **Real-time Updates:** WebSocket or polling for live data
5. **Error Handling:** Comprehensive error recovery

Environment Configuration

Create `.env.local` for frontend:

```
NEXTAUTH_SECRET=your-secret-key
NEXTAUTH_URL=http://localhost:3000
DATABASE_URL=your-database-url
```

Create environment variables for backend:

```
OPENAI_API_KEY=your-openai-key
GROK_API_KEY=your-grok-key
GOOGLE_API_KEY=your-google-key
OPENROUTER_API_KEY=your-openrouter-key
```

Installation & Setup Instructions

Prerequisites

- Node.js 18+ and Yarn
- Python 3.9+ with asyncio support
- Docker (for Ollama local models)

Step-by-Step Setup

1. **Clone and Install:**

```
```bash
git clone
cd dspy_boss_dashboard
```

```
Frontend setup
cd app && yarn install
```

```
Backend setup
cd .. && pip install -r requirements.txt
```



- Configuration:
  - Set up environment variables
  - Configure LLM provider API keys
  - Initialize database schema
- Run the System:

```
```bash
# Start frontend
cd app && yarn dev
```


Backend runs automatically via integration layer
```


```

UI Design Specifications

Color Scheme

- **Background:** Deep dark (hsl(0, 0%, 3.9%))
- **Cards:** Glass effect with 10% white opacity
- **Primary:** Healthy green (hsl(142, 76%, 36%))
- **Secondary:** Deep blue-gray (hsl(210, 40%, 8%))
- **Accent:** Bright highlights for status indicators

Component Styling

- **Glass Cards:** Backdrop blur with subtle borders
- **Typography:** Inter font with proper hierarchy
- **Animations:** Subtle hover effects and transitions
- **Icons:** Lucide React with consistent sizing
- **Spacing:** Consistent padding and margins

Key Technical Considerations

DSPY Integration

- Use `dspy.Predict()` for signature execution, not direct instantiation
- Implement proper signature classes with `InputField` and `OutputField`
- Handle model configuration and provider switching

Error Handling

- Implement lazy initialization for async components
- Handle missing dependencies gracefully
- Provide meaningful error messages and recovery

Performance

- Use efficient state caching
- Implement proper async patterns

- Optimize database queries
- Monitor resource usage

Security

- Secure API key management
- Proper authentication and authorization
- Input validation and sanitization



Dashboard Features

Main Dashboard

- System overview with key metrics
- Real-time agent status grid
- Performance trends and forecasting
- Active task distribution

Agent Management

- Agent hierarchy visualization
- Agent spawning controls
- Performance monitoring
- Communication logs

Boss Interface

- Direct chat with Boss Agent
- Strategic planning interface
- System directive configuration
- Autonomous operation controls

Analytics

- System performance metrics
- Task completion analytics
- LLM provider usage statistics
- Error rate monitoring



MCP Server Integration

Implement MCP servers for:

- **News Fetching**: Daily news aggregation
- **YouTube Analytics**: Content scraping and analysis
- **Custom Tools**: Extensible tool system



Special Features

Autonomous Operation

- Self-starting execution loops
- Intelligent decision making
- Adaptive behavior based on performance

- Error recovery and resilience

Agent Coordination

- Hierarchical task delegation
- Inter-agent communication
- Dynamic agent spawning
- Load balancing

State Intelligence

- Historical pattern recognition
- Future state forecasting
- Performance prediction
- Optimization suggestions



Implementation Notes

Critical Technical Details

1. **Pydantic Models:** Avoid `model_config` field name (conflicts with Pydantic v2)
2. **DSPY Signatures:** Use `dspy.Predict(Signature)` not `Signature()`
3. **Async Initialization:** Use lazy initialization for event loop components
4. **Import Structure:** Maintain proper relative imports in backend
5. **Package Management:** Use yarn for frontend, pip for backend

Frontend Requirements

- App Router structure with proper `page.tsx` files
- Shadcn UI components with custom theming
- Proper TypeScript configuration
- Authentication integration
- Real-time data updates

Backend Requirements

- Proper Python package structure with `init.py` files
- DSPY signature implementations
- Async/await patterns throughout
- Comprehensive logging
- Error handling and recovery



Deployment Considerations

Build Requirements

- Ensure `yarn.lock` exists for consistent builds
- Proper environment variable configuration
- Database initialization scripts
- Health check endpoints

Production Features

- Automatic error recovery

- Performance monitoring
- Resource management
- Security hardening

This prompt provides the complete blueprint for recreating the DSPY Boss autonomous AI system with all its sophisticated features and luxury UI design.